

GODROP

Software Technical Documentation

Architecture, data model, API contract, and per-app implementation reference

Version: 1.0

Date: 31 May 2026

Repository: godrop (Turborepo + pnpm workspaces monorepo)

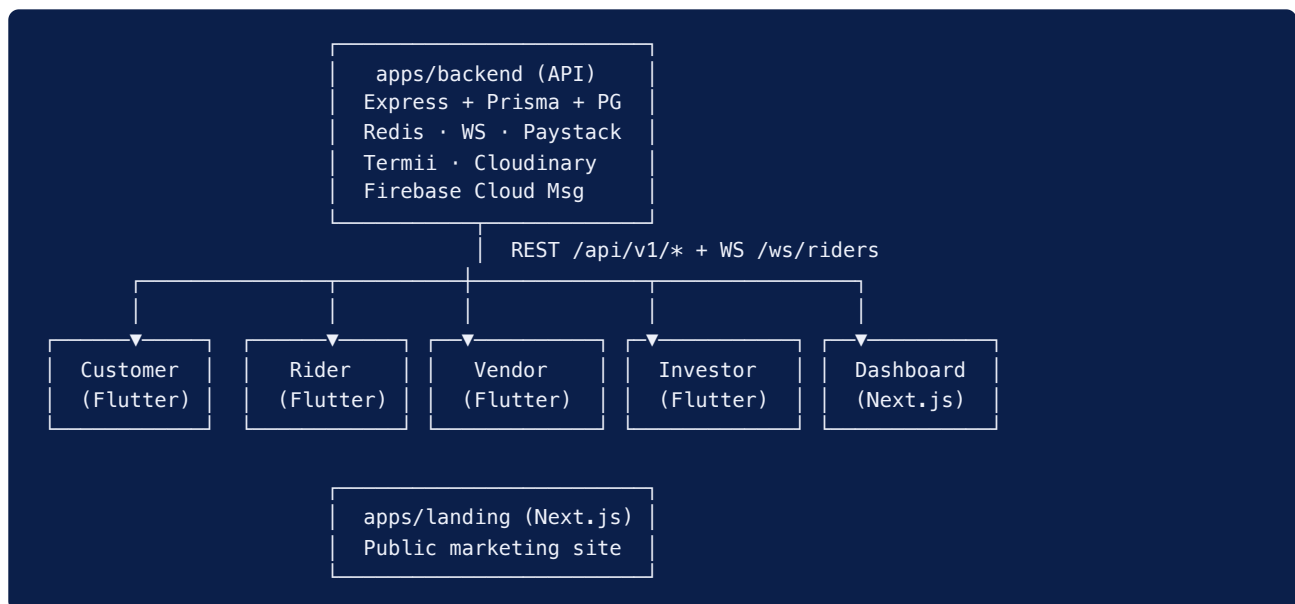
Audience: Engineers onboarding to, or integrating with, any Godrop app or the backend API

Contents

1. System Architecture
2. Monorepo Layout & Tooling
3. Backend API (@godrop/backend)
4. Data Model
5. API Surface Overview
6. Ops Dashboard (@godrop/dashboard)
7. Landing Site (@godrop/landing)
8. Mobile Apps (Flutter)
9. Cross-Cutting Conventions
10. Environments & Deployment

1. System Architecture

Godrop is a single Turborepo monorepo containing one backend service, two Next.js web apps, and four independent Flutter mobile apps, all consuming a single REST + WebSocket API described by one OpenAPI contract.



The backend is the sole source of truth. All five client apps (four Flutter apps + the dashboard) talk to the same Express API and share the same OpenAPI-defined contract at `apps/backend/openapi.yaml`. The landing site is standalone and does not require backend auth for its public pages, but links to app download and API-backed contact/business forms.

2. Monorepo Layout & Tooling

```
godrop/  
├── apps/  
│   └── backend/      # Node.js + Express API (@godrop/backend)
```

```

├── mobile/
│   ├── Customer/ # Flutter – customer-facing app
│   ├── Rider/   # Flutter – rider-facing app
│   ├── Vendor/  # Flutter – vendor admin app
│   └── Investor/ # Flutter – GoDrop Invest app
├── landing/     # Next.js public marketing site (@godrop/landing)
├── dashboard/   # Next.js internal ops dashboard (@godrop/dashboard)
├── packages/
│   ├── shared-types/ # TypeScript types shared across all JS/TS apps
│   └── ui/           # (future) Shared React component library
├── turbo.json
├── pnpm-workspace.yaml
└── package.json

```

Concern	Choice
Monorepo tooling	Turborepo + pnpm workspaces (pnpm only — never npm/yarn)
Language (JS/TS apps)	TypeScript everywhere
Language (mobile)	Dart / Flutter
API contract	<code>apps/backend/openapi.yaml</code> — the single contract all apps integrate against
Shared types	<code>packages/shared-types</code> , mirrored manually as <code>Dart freezed</code> / <code>json_serializable</code> models in each Flutter app

Recommended build order

1. `packages/shared-types` — core types first
2. `apps/backend` — API + `openapi.yaml`
3. `apps/dashboard` — most complex frontend, benefits from full API context
4. `apps/landing` — simpler, reuses dashboard patterns
5. `apps/mobile/*` — generated/hand-mirrored from `openapi.yaml`

Running everything

```

pnpm install      # install all workspaces
pnpm dev          # run all apps in parallel via Turborepo
pnpm dev:backend  # backend only (port 4000)
pnpm dev:landing  # landing only
pnpm dev:dashboard # dashboard only (port 3001)

```

3. Backend API (@godrop/backend)

3.1 Tech stack

Layer	Choice
Runtime	Node.js 20 + TypeScript
Framework	Express.js v4
Database	PostgreSQL via Prisma ORM 5.x
Cache / realtime	Redis (ioredis)

Auth	JWT (access + refresh token pattern for phone-auth apps; single long-lived JWT for the vendor app)
Validation	Zod schemas in <code>src/validators/</code>
Payments	Paystack (Nigerian gateway — not Stripe)
SMS / OTP	Termii (Nigerian SMS provider), abstracted behind <code>services/sms/smsProvider.ts</code>
Maps	Google Maps Platform — Directions, Geocoding, Distance Matrix
File uploads	Cloudinary (via <code>multer</code> + <code>cloudinaryService.ts</code>)
Push notifications	Firebase Admin (FCM) via <code>fcmService.ts</code>
Email	<code>nodemailer</code> via <code>emailService.ts</code>
PDF export	<code>pdfkit</code> (audit log export)
Real-time	<code>ws</code> — WebSocket server for rider location streaming (<code>wss://.../ws/riders</code>)
API docs	<code>swagger-ui-express</code> serving <code>openapi.yaml</code>

3.2 Project structure

```
apps/backend/src/
  index.ts           # Express app entry point
  routes/            # Route registration only — no business logic
  controllers/       # Request/response handling per route
  services/          # Business logic + external integrations
  middleware/        # Auth, validation, error handling
  utils/             # geocode, formatKobo, generateTrackingCode, vendorHours...
  validators/        # Zod request schemas
  types/             # Backend-local TS types
  prisma/
    schema.prisma    # Single Prisma schema (1,247 lines, 51 models, 26 enums)
    seed.ts
```

3.3 Layering convention

Route files only register routes; request handling lives in controllers; all DB access and third-party calls live in services. All errors are caught and passed to `next(err)` — never `res.send` directly from a catch block. This keeps error formatting centralized in the error-handling middleware.

3.4 Route modules

Route file	Domain
<code>auth.ts</code>	Customer/user registration, OTP, login, refresh, password reset
<code>me.ts</code>	Authenticated customer profile, addresses, wallet, favorites, notifications
<code>food.ts</code> / <code>grocery.ts</code> / <code>retail.ts</code> / <code>pharmacy.ts</code>	Vendor listing, search, product browse, checkout per category
<code>orders.ts</code>	Cross-category order detail, cancel, tracking, reorder, rating
<code>parcel.ts</code>	Parcel quote, vehicle types, order creation (multi-dropoff)
<code>truck.ts</code>	Truck quote, apartment types, pricing config, order creation
<code>payments.ts</code>	Paystack initialize/verify
<code>promotions.ts</code>	Promo code application, marketing banners

<code>riderApp.ts</code>	Rider auth, jobs, active order transitions, earnings, wallet, KYC, notifications
<code>vendorAdmin.ts</code>	Vendor-app auth-protected endpoints: orders, catalog, team, wallet, settings, analytics, audit-log export
<code>vendorOnboarding.ts</code>	Public vendor application intake (<code>POST /vendor</code> , multipart)
<code>vendorWallet.ts</code>	Vendor wallet bank resolution and Naira-denominated withdrawal
<code>businessAdmin.ts</code>	Business-scoped admin — team/rider assignment for multi-outlet businesses
<code>systemAdmin.ts</code>	Platform-wide admin: vendors, riders, customers, orders, admins, messaging, push, analytics, audit logs, heroes
<code>investor.ts</code>	GoDrop Invest: auth, assets, plans, investments, wallets, Flow Safe, KYC, bank accounts
<code>heroes.ts</code>	Public marketing hero content for the customer app home screen
<code>search.ts</code>	Cross-category search
<code>contact.ts</code>	Public contact form intake (landing site)
<code>config.ts</code>	Public + per-vendor runtime config (e.g. payment options)

3.5 Environment

Copy `.env.example` to `.env` and fill in values. Secrets are never hardcoded and `.env` is never committed.

3.6 Running locally

```
pnpm dev      # ts-node-dev with hot reload on port 4000
pnpm build    # prisma generate + tsc → dist/
pnpm start    # prisma migrate deploy + node dist/index.js
pnpm db:migrate # prisma migrate dev
pnpm db:seed  # tsx src/prisma/seed.ts
```

4. Data Model

The Prisma schema (`apps/backend/src/prisma/schema.prisma`) defines **51 models** and **26 enums**. Grouped by domain:

4.1 Identity & auth

`User` , `Admin` , `Vendor` , `Rider` , `Investor` — five distinct principal types, each with its own auth flow and token model. `RefreshToken` , `RiderRefreshToken` , `InvestorRefreshToken` are separate tables per principal (the vendor app deliberately has no refresh token). `Otp` backs phone verification for customers, riders, and investors.

4.2 Commerce

`Vendor` , `ProductCategory` , `Product` — vendor catalog. `Order` , `OrderItem` , `OrderEvent` — the universal order model shared by all six order types, with `OrderEvent` providing an append-only status-change timeline used for tracking and the dashboard order-detail view. `ParcelDropoff` , `ParcelVehicleType` , `ApartmentType` , `TruckType` , `TruckPricingConfig` — category-specific extensions to the base order.

4.3 Money

Wallet / WalletTransaction (customer), VendorWallet / VendorWalletTransaction / VendorBankAccount / VendorWithdrawal (vendor), RiderEarning / RiderWithdrawal (rider), BusinessWallet / BusinessWalletTransaction (multi-outlet business owners), InvestorWallet / InvestorTransaction / InvestorBankAccount / FlowSafeVault (investor). SavedCard stores tokenized Paystack card references for repeat charges.

4.4 Investment product

InvestmentAsset (a fundable vehicle), InvestmentPlan (term/rate template), Investment (an investor's position in an asset under a plan), InvestmentPayout (scheduled monthly payout rows), InvestorSettings .

4.5 Ops & platform

Business (multi-outlet vendor parent entity), PlatformSettings (singleton config row, e.g. riderEarningRate), Promotion , Banner , Hero , ContactMessage , Review , Favorite , SavedAddress , AuditLog (every sensitive admin/vendor action), Notification / AdminNotification / RiderNotification / InvestorNotification and matching *PushToken tables per principal.

4.6 Core enums

Enum	Values
OrderType	FOOD · GROCERY · RETAIL · PHARMACY · PARCEL · TRUCK
OrderStatus	PENDING · ACCEPTED · PREPARING · READY_FOR_PICKUP · PICKED_UP · IN_TRANSIT · DELIVERED · CANCELLED · FAILED
PaymentMethod	CARD · WALLET · CASH · WALLET_CARD
PaymentStatus	PENDING · PAID · REFUNDED · FAILED
VendorType	RESTAURANT · GROCERY · RETAIL · PHARMACY
VendorStatus	PENDING · APPROVED · REJECTED · SUSPENDED
UserStatus	customer account state (active/suspended, etc.)
RiderKycStatus , RiderWithdrawalStatus , RiderEarningStatus	rider onboarding & payout lifecycle
InvestmentStatus , InvestmentAssetStatus , InvestmentPayoutStatus	investment lifecycle
InvestmentVehicleClass	asset class (Keke, shuttle, ride car, bike)
InvestorKycStepStatus , InvestorTxType , InvestorTxStatus	investor KYC and transaction states
FlowSafeVaultKind , FlowSafeVaultStatus	flexible vs. locked interest-bearing vaults
VendorWalletTxType , VendorWithdrawalStatus , BusinessWalletTxType , WalletTxType	per-principal wallet transaction typing
AdminRole , AdminType	system vs. business-scoped admin access

4.7 Order model detail

```

Order {
  id, trackingCode (unique), customerId → User, type: OrderType,
  status: OrderStatus, vendorId? → Vendor, truckTypeId? → TruckType,
  parcelVehicleTypeId? → ParcelVehicleType, apartmentTypeId? → ApartmentType,
  numLoaders?, ... payment + pricing fields, timestamps
}
OrderItem { orderId → Order, productId? → Product, name, unitPriceKobo, quantity, totalKobo, notes }
OrderEvent { orderId → Order, status change history – powers tracking timeline }

```

A single nullable-foreign-key design (`vendorId?` , `truckTypeId?` , `parcelVehicleTypeId?` , `apartmentTypeId?`) lets one `Order` table serve all six order types without a class-table-inheritance split — the `type` discriminator determines which foreign keys are populated.

5. API Surface Overview

The full contract is `apps/backend/openapi.yaml` (243 documented paths) — always the source of truth. Grouped by prefix:

Prefix	Consumer	Purpose
<code>/auth/*</code>	Customer app	Register, OTP request/verify, login, refresh, logout, password reset
<code>/me/*</code>	Customer app	Profile, addresses, avatar, favorites, wallet, notifications, push token
<code>/food/*</code> , <code>/grocery/*</code> , <code>/retail/*</code> , <code>/pharmacy/*</code>	Customer app	Store/product browse, search, checkout per category
<code>/orders/*</code>	Customer app	Order detail, cancel, tracking, reorder, rating
<code>/parcel/*</code> , <code>/truck/*</code>	Customer app	Quote + book, vehicle/apartment type catalogs (admin sub-paths for config)
<code>/payments/*</code>	Customer + Investor apps	Paystack initialize/verify
<code>/promotions/*</code> , <code>/heroes</code> , <code>/search</code> , <code>/config</code> , <code>/contact</code>	Customer app / Landing	Marketing content, cross-category search, public config, contact intake
<code>/rider/*</code>	Rider app	Auth, profile, KYC, availability, location, jobs, order transitions, earnings, withdrawals, notifications
<code>/vendor</code>	Vendor app (public)	New-vendor onboarding application
<code>/vendor-admin/*</code>	Vendor app (authed)	Orders, categories, products, team, settings, wallet, analytics, audit-log export
<code>/business-admin/*</code>	Vendor app (OWNER/business)	Business profile, riders (multi-outlet delivery), team, wallet
<code>/investor/*</code>	Investor app	Auth, assets, plans, investments, wallet, Flow Safe, KYC, bank accounts, notifications, settings, support
<code>/admin/*</code>	Dashboard	System-wide: businesses, customers, riders, vendors, orders, admins, analytics, audit logs, messaging, push, heroes

Real-time rider location streams over a WebSocket at <wss://api.naijagodrop.com/ws/riders> , consumed by both the customer app's tracking screen and the dashboard's live rider map.

6. Ops Dashboard (@godrop/dashboard)

6.1 Tech stack

Layer	Choice
Framework	Next.js 14 (App Router only — no <code>pages/</code>)
Styling	Tailwind CSS + shadcn/ui + Radix primitives
Tables	TanStack Table v8 (sortable, filterable, paginated)
Charts	Recharts
Forms	React Hook Form + Zod
Auth	NextAuth.js, admin-only role
Maps	Google Maps JS API (live rider tracking view)
Global state	Redux Toolkit
Data fetching	RTK Query + SWR (auto-revalidating live panels)
Deployment	Vercel — <code>ops.godrop.ng</code> , separate from landing

6.2 Structure

```
apps/dashboard/  
  app/           # routes: orders, riders, vendors, businesses, customers,  
                 # trucks, parcels, analytics, disputes, messaging, push,  
                 # admins, heroes, settings, catalog, team, wallet, login...  
  components/    # ui/ (shadcn primitives), dashboard/, layout/, catalog/, providers/  
  features/      # one folder per app/ route — business logic co-located  
  store/         # Redux slices + RTK Query API services  
  lib/           # utilities, api client  
  types/         # dashboard-local TS types
```

6.3 Conventions

- All data fetching happens in Server Components or SWR hooks — never directly inside render.
- `@godrop/shared-types` is the only source of shared domain types — never redefine `Order` , `User` , etc. locally.
- Monetary values are always displayed in Naira, formatted from Kobo via a shared `formatNaira(kobo)` util.
- All destructive/irreversible admin actions (cancel order, deactivate rider) require a confirmation dialog.
- Active-orders panel polls every 10s via SWR `refreshInterval` ; new-order alerts use the browser Notification API.

6.4 Environment

```
NEXT_PUBLIC_API_URL=http://localhost:4000/api/v1  
NEXTAUTH_SECRET=your_nextauth_secret  
NEXTAUTH_URL=http://localhost:3001  
NEXT_PUBLIC_GOOGLE_MAPS_API_KEY=your_google_maps_key
```


7. Landing Site (@godrop/landing)

Layer	Choice
Framework	Next.js 16 (App Router)
Styling	Tailwind CSS v4 + shadcn/ui
Animation	Framer Motion, GSAP, Remotion (video), Lenis (smooth scroll)
Forms	React Hook Form + Zod
Deployment	Vercel, separate project from the dashboard

Routes: home, about, contact, business, vendors, riders, privacy-policy, terms-of-service, community-guidelines, reset-password (email deep-link landing). Path alias `@/*` maps to repo root. Rules: move repeated UI into reusable components, keep side effects out of UI components, prefer server-side data fetching unless client interactivity is required.

8. Mobile Apps (Flutter)

Four independent Flutter projects under `apps/mobile/`, sharing the same architectural conventions but each its own app with its own entry point, auth model, and money convention.

8.1 Shared conventions (all four apps)

Concern	Choice
State management	<code>flutter_bloc</code> — <code>Bloc</code> for multi-event flows, <code>Cubit</code> for simple state; one Bloc/Cubit per feature, never emitted from outside
Navigation	<code>go_router</code> — no raw <code>Navigator.push</code> in feature code
HTTP	Dio (+ retrofit/codegen in Vendor and Investor apps) with JWT-refresh interceptors
Local storage	<code>flutter_secure_storage</code> (tokens), <code>shared_preferences</code> (settings/session cache)
Models	<code>freezed</code> + <code>json_serializable</code> , mirroring <code>@godrop/shared-types / openapi.yaml</code>
Money display	Kobo → Naira with ₦ symbol via shared <code>formatKobo()</code> / <code>formatNaira()</code> utils
Phone input	🇳🇬 flag, auto-prepend <code>+234</code> , validate 11-digit local format
API layer	All calls in <code>lib/shared/api/</code> service classes — never call Dio directly from widgets

8.2 Customer app

Entry: `lib/main_customer.dart`. Auth: phone+OTP, access+refresh JWT pair. Maps: Google Maps Flutter + Geolocator. Tracking: WebSocket via `web_socket_channel`. Payments: Paystack Flutter SDK. Structure: `features/{auth,home,orders,tracking,truck,profile}/bloc/`.

8.3 Rider app

Entry: `lib/main_rider.dart`. Auth: phone+OTP (shared pattern with customer app). FCM is the primary channel for new-job alerts. Structure: `features/{auth,rider/jobs,rider/active,rider/history,profile}/bloc/`. Battery-aware GPS polling — reduced frequency when no job is active.

8.4 Vendor app

Entry: `lib/main.dart` . Differs from the other three apps: email+password login at `POST /api/v1/admin/login` , a single 7-day JWT with **no refresh token**, and role-gated UI (`SessionCubit : isOwner , isManagerOrAbove`) mirrored by server-side enforcement.

Money convention split (read carefully): Catalog, orders, and analytics endpoints speak integer **Kobo** (`priceKobo , totalKobo , revenueKobo`) → `formatKobo()` . Wallet endpoints speak **Naira decimals** (`balance , amount ; withdraw minimum ₦100`) → `formatNaira()` . This is the one deliberate exception to the platform-wide Kobo rule.

New vendors apply via unauthenticated multipart `POST /vendor` (business info, lat/lng, owner account, CAC registration + government ID ≤10MB, optional utility bill) — created as `PENDING` ; the owner can sign in immediately, but the store is unlisted until system-admin approval. Opening hours (`shared/models/opening_hours.dart`) mirror `apps/backend/src/utils/vendorHours.ts` : canonical keys `sun – sat , {open, close}` in HH:mm Africa/Lagos time, `null` = closed that day, overnight ranges supported (`close <= open`), an absent map means "no restriction" — never send `{}` to clear hours.

Structure:

`features/{splash,auth,onboarding,dashboard,orders,catalog,wallet,team,settings,profile}/` .

Codegen: `dart run build_runner build --delete-conflicting-outputs` after changing `shared/models/` or `shared/api/services/` .

8.5 Investor app — GoDrop Invest

Entry: `lib/main.dart` . Design source: a 30-screen Claude.ai design project (`GoDrop Invest – App.html`) — **ships light mode only**.

Auth: register (`POST /investor/auth/register`) → phone OTP verify → access+refresh pair; login by email or phone + password; a `QueuedInterceptor` auto-refreshes on 401 and retries once.

Money: everything is integer Kobo (`amountKobo , balanceKobo`) — no Naira-decimal endpoints, unlike the vendor app. Rates come as basis points (`projectedRoiBps , monthlyRateBps`) formatted with `formatBps()` .

Domain glossary: *Asset* — a fundable vehicle (KEKE / SHUTTLE / RIDE_CAR / BIKE). *Plan* — Starter (6mo) / Rider (12mo) / Fleet (18mo), sets lock period + monthly rate. *Investment* — amount × plan on an asset, generates an `InvestmentPayout` schedule, capital returns at maturity. *Wallets* — Main (invest from), Payout (returns land here), Flow Safe (interest-bearing vaults, flexible or locked).

Structure:

`features/{splash,onboarding,auth,home,assets,invest,portfolio,wallet,safe,transactions,profile}/` .

Bottom tabs: Home · Safe · My Assets · Transaction · Profile (`MainShell`). Codegen: same `build_runner` command as the Vendor app.

8.6 Nigerian UX considerations (all mobile apps)

- Support low-bandwidth conditions — lazy-loaded images, paginated lists.
- Address input surfaces popular Lagos areas (Ikeja, VI, Lekki, Yaba, Surulere).
- ETAs shown in minutes, never exact clock times.
- Cash-on-delivery is a valid, first-class payment option.
- Dark mode: nice-to-have, not required at launch (Investor app is light-only by design).

9. Cross-Cutting Conventions

9.1 Money

Store and transmit all monetary values as **integer Kobo** — never floats. The single documented exception is the vendor app's wallet endpoints (Naira decimals). Investor rates use basis points, not percentages, on the wire.

9.2 Phone numbers

Canonical format is E.164: `+234XXXXXXXXXX`. Every client normalizes local `0803...` input before sending to the API.

9.3 API response envelope

All backend responses follow a shared `ApiResponse<T>` shape (from `@godrop/shared-types`): `{ success, ...payload }` on success, `{ success: false, error }` on failure. Flutter apps parse errors via a shared `parseDioError` helper local to each app's auth module.

9.4 Auth models by principal

Principal	Login	Tokens
Customer	Phone + OTP	Access + refresh JWT pair
Rider	Phone + OTP	Access + refresh JWT pair
Vendor admin	Email + password	Single 7-day JWT, no refresh
Investor	Register+OTP, or email/phone + password login	Access + refresh JWT pair, auto-refresh interceptor
Dashboard admin	Email + password via NextAuth	NextAuth session

9.5 Error handling

Backend: every error must reach centralized handling via `next(err)`. Frontend: no silent failures — all mutations that matter (order cancel, rider deactivation) require an explicit confirmation UI before firing.

10. Environments & Deployment

App	Dev URL	Prod	Deploy target
Backend	<code>http://localhost:4000</code>	<code>api.naijagodrop.com</code>	Node host (PM2/Docker — <code>pnpm build</code> → <code>pnpm start</code> runs <code>prisma migrate deploy</code>)
Dashboard	<code>http://localhost:3001</code>	<code>ops.godrop.ng</code>	Vercel
Landing	<code>http://localhost:3000</code>	<code>godrop.ng</code> (public)	Vercel (separate project from dashboard)
Customer / Rider / Vendor / Investor	Android emulator: <code>http://10.0.2.2:4000/api/v1</code>	<code>https://api.naijagodrop.com/api/v1</code>	Play Store / TestFlight; APKs also hosted on the landing site

10.1 Launch geography

Product goal is nationwide Nigeria coverage. Launch market is Lagos, used to prove the model. States targeted for the next expansion wave: Abuja, Port Harcourt, Ibadan, Kano.